GVS5H: Zero-Shot Self-Orchestration With Ledger-Based Control Improves Coding in Language Models

Victor Gao, Vida Khosrowshahi, Ali Khosrowshahi, Xihao Sun, Juhyun Lee, Ethan Tran, and Simon (Sang Won) Lee

***Abstract*—Frontier coding performance is typically attained with large, costly proprietary models. We introduce ledger-based zero-shot self-orchestration, a training-free method in which fresh instances of one model decompose problems and coordinate through a shared file system. A single model acts both as a manager that writes a plan and maintains a task list and as workers that each execute one assigned task. Every call starts from an empty context and reads state only from the shared ledger files. Across eleven open and closed-weight models on the 100 latest hard LiveCodeBench problems the method yields as much as 25.6 points improvement, boosting several cheaper models to frontier-level performance. Orchestrated Qwen3.8 Flash Next scores 93.0% against Fable 5's 90.4% at 9% of the cost, while the smaller Qwen3.8-27B reaches 92.4%. Gains are not universal as some models are unchanged or worse. Transcript analysis attributes the gain to decomposition and persistent context. Inference-time organization can reach or exceed frontier coding accuracy at a fraction of the cost on self-hostable weights.**

***Index Terms*—Code generation, large language models, multiagent systems, self-orchestration.**

I. Introduction

LARGE language models (LLMs) are increasingly deployed not as single responses to a prompt. Instead, they are wrapped in a scaffold that performs multiple model calls, often interlaced with planning, tool calling, note taking, and revision over multiple turns, on the intuition that decomposing a problem and making incremental progress should beat an attempt at answering everything in one pass.

In this paper, we test and quantify this intuition by measuring the performance of a single model call against a basic system consisting of multiple alternating orchestration and execution calls (manager and worker roles). Each call has a new context and communicates through a shared filesystem, with no access to additional tools.

We call the method studied here zero-shot self-orchestration (GVS5H: Gorilla vs. 5 Humans): the orchestrator is neither trained for orchestration nor given task-specific demonstrations of how to decompose or coordinate the problem. Every model role is the same model in a fresh context with a short generic prompt. This scaffold is further described in Section II.

A. Related Work

Our test uses existing proprietary and open-weight language models, and what we vary is the coordination layer above them. Prior systems differ mainly in how that layer is decided. One line trains the coordinator: Fugu [1] learns to assemble and direct a pool of expert workers per query, and Conductor [2] is a 7B model trained with reinforcement learning to design worker communication topologies; earlier query routers learn to pick a model per query [3]. A second fixes roles and hand-off order in advance, so every problem flows through the same pipeline [4]–[8], while frameworks such as AutoGen [9] instead supply configurable conversable agents and leave the conversation pattern open; Trinity [10] sits at the boundary with the first, fixing a three-role workflow but learning which model fills each role. A third coordinates through a shared store rather than a pipeline - ARIADNE [11] pairs a blackboard with Monte Carlo tree search (MCTS) on program generation, LbMAS [12] mediates all communication through a global board - or decentralizes entirely, through debate [13] or sampling and voting [14]; Mixture-of-Agents [15] instead aggregates in layers. A separate line asks not how coordination is decided but whether reported gains survive equalizing test-time compute [16], [17].

Ours is the training-free counterpart to the first family and the dynamic counterpart to the second. The manager is a fixed prompt with no learning, which lets us ask how much of the orchestration benefit is available without training, and when it materializes. The list of tasks is dynamically generated and updated throughout the problem solving process rather than predetermined. Its workspace is a shared store like ARIADNE's blackboard, but coordination is done by the manager rather than through MCTS, reward models, or learned search policies. We do not attempt an equal-token comparison across the same model - the loop necessarily spends more than a single call. Instead, we ask whether that spend buys better solutions, and whether it is more cost-effective than moving to a larger model.

II. Method

Every role but the verifier is the same underlying model, called in a fresh context and coordinating through a shared workspace on disk:

`<ws>/task.md`	the problem statement
`<ws>/plan.md`	the manager's overarching plan
`<ws>/tasks.json`	a debug mirror of the task list
`<ws>/notes.md`	curated ideas / findings / partial proofs
`<ws>/solution.py`	the current solution

Workers never talk to each other; each sees the ledger plus the one task the manager assigns. The control flow (Fig. 1) is:

1) **Manager - plan.** The manager reads the problem and writes a 3–6 sentence strategy plus 3–6 concrete seed tasks.

Victor Gao, Vida Khosrowshahi, and Ali Khosrowshahi contributed equally to this work. *(Corresponding author: Simon (Sang Won) Lee.)*

V. Gao, V. Khosrowshahi, A. Khosrowshahi, X. Sun, E. Tran, and S. Lee are with Persis Capital Inc., Toronto, ON, Canada (e-mail: slee@persisholdings.com).

J. Lee is with DRI Healthcare, Toronto, ON, Canada.

2) **Worker - brainstorm.** The first worker does not write a solution; it identifies the core difficulty, lists candidate approaches and pitfalls, and appends them to `notes.md`, proposing next steps.
3) **Manager - manage.** The manager folds the plan and the brainstorm into one task list (merging duplicates, marking completed items, adding new sub-tasks), then either declares the problem done or names the single next task.
4) **Worker - execute.** A fresh worker executes that one task and rewrites both `solution.py` and `notes.md` (the notes are replaced by a curated whole rather than appended to), then proposes remaining steps.
5) **Verifier - run the public tests.** Whenever the round's worker produced a fresh candidate, the program is executed against the problem's public stdin tests where it has them. The pass/fail verdict, with the first failing case, is fed back to the manager and treated as ground truth: a failing run overrides a done verdict, forcing the loop to continue with a fix-or-switch task. Control returns to the manager (step 3).
6) **Finalizer.** A finalization worker emits the definitive solution whenever the loop ends without a clean sign-off - the round budget is spent, the manager reissues a task it just handed out, or it names no task at all. The call is skipped when the manager declares the problem done and a usable solution is already on disk.

Guards: a round budget of 10 manager→worker cycles (`MAX_ITERS`); a no-progress guard (if the manager reissues the exact task it just handed out, the loop stops); and a cut-off summarizer: a worker that hits the token cap mid-attempt has its partial reasoning summarized by a fresh short call so its ideas still reach the manager. The plan and notes files are capped when written, at 4,000 and 8,000 characters, so neither grows without bound across rounds.

A. Baseline

The single-call baseline is the same model given the same problem in exactly one call, with no workspace, loops, or other roles. The single call receives the solver system prompt verbatim; the manager arm's task workers receive that same prompt wrapped in a subagent preamble, and their user message additionally carries the plan, the current notes and the current solution. The two arms differ only in that scaffold. Section II-C gives the per-role temperatures.

B. The Original Scaffold

The loop above is v2, which produced the pinned-backend results and is drawn in Fig. 1. The original scaffold produced the OpenRouter-served set of Section III-D. It runs the same loop on closely matched prompts, with five differences, all of them on the manager arm: it allows 4 manager→worker cycles rather than 10 (`MAX_ITERS`), it has no public-test verifier at step 5, it has no cut-off summarizer, its workers append to `notes.md` rather than rewriting it, and the plan and notes files feeding a prompt are unbounded rather than capped.

C. Experimental Setup

1) Benchmark: LiveCodeBench code-generation, release_v6 [18]: the 100 most recent problems in the hard split by contest date, which limits training-data contamination [18]. We refer to this set as LCB-100. Solutions are graded by LiveCodeBench's own hidden-test evaluator, after the two corrections described in Section I of the supplementary material; every number below is post-correction. The verifier of step 5 runs only stdin-format public tests, which 73 of the 100 problems carry; the other 27 are LeetCode-style, with call-based public tests the engine does not run, so those get no verifier signal and are checked by the hidden-test grader alone.

2) Models and serving: Eleven model configurations: Qwen3.5-9B [19] (9B), Qwen3.6-35B-A3B [20] (35B total, 3B active per token), Qwen3.8-27B [21] (27B, open weights, served locally in 8-bit floating point (FP8)), Qwen3.8-Flash-Next [22] (125B total, 6B active per token, open weights, served locally in FP8), DeepSeek-V4.1-Flash [23] (552B total, 16B active per token at decode, open weights, served by the first-party DeepSeek application programming interface (API)), MiniMax-M3 [24] (428B total, $\sim$23B active), Kimi-K3 [25] ($\sim$2.8T total, $\sim$16 of 896 experts active per token), and the closed frontier models Opus-5 [26], the two GPT-5.6 variants Terra and Luna [27], and Claude Fable 5 [26], all four of undisclosed size. Serving splits the set in two. The six pinned-backend models are each served on one fixed backend for v2 comparison, because gateway routing proved the dominant source of run-to-run noise (Section IV): Terra and Luna by the OpenAI API, Qwen3.8-27B and Qwen3.8-Flash-Next by our own vLLM, DeepSeek-V4.1-Flash by the first-party DeepSeek API, and Fable 5 by the Anthropic Messages API. The five OpenRouter-served models ran through a single OpenAI-compatible gateway on the original scaffold, and are reported separately in Section III-D because both the serving path and the scaffold version differ. Fable 5 was run single-only, so it contributes no manager delta.

3) Temperatures: The manager arm's generative calls use slightly higher temperatures, where the work is generative rather than executive: 0.3 to write the plan and 0.4 to brainstorm, against 0.2 for task execution and for curating the task list. The single-call baseline runs at 0.2, matching the workers. These settings are identical in both scaffold versions.

4) Conditions: The pinned-backend set runs at one setting (128k output cap, reasoning on, $\times 5$ passes) with the five models in both arms. The cap is a per-call `max_tokens` bound, not a context-window limit. The OpenRouter-served set runs at three: 128k reasoning-on $\times 1$; 16k reasoning-off $\times 5$; and 128k reasoning-off $\times 1$ as a control isolating the small cap; Section II of the supplementary material gives the purpose of each. Appendix A gives the reasoning-effort settings per provider, the instrumentation, and the cap-matching replay used for Qwen3.8-27B's single arm.

5) Statistics: Reported $\pm$SD is the sample standard deviation (SD; divisor $n - 1 = 4$) of the five pass scores. Five-pass comparisons use a paired sign-flip permutation test on per-problem mean Δ ($n = 100$ problems; 2×10^5 resamples), Holm-corrected where a family of models is tested; 95% t-intervals for mean pass@1 across the five passes ($\mathrm{df} = 4$).

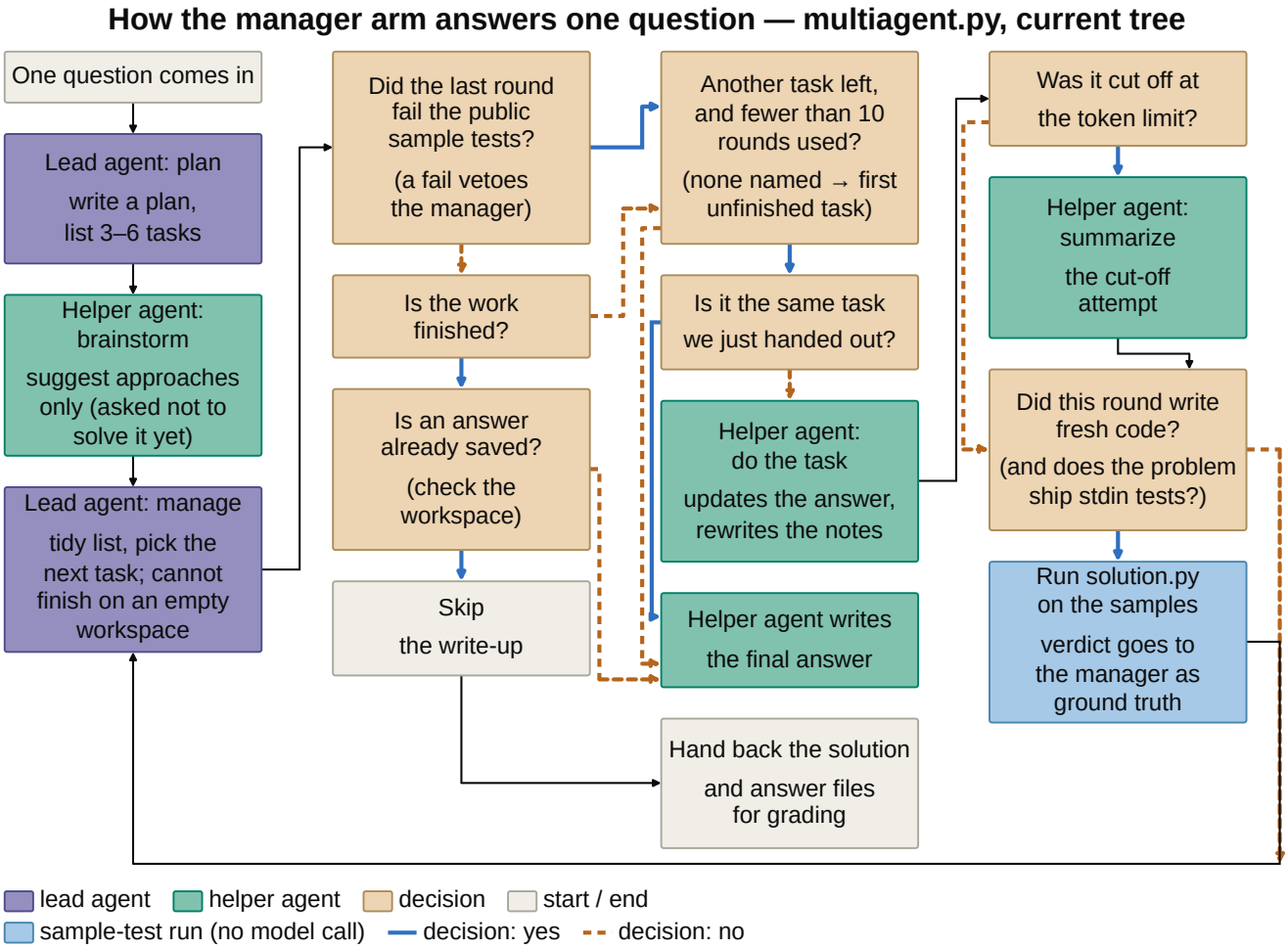


Fig. 1. Manager–worker control flow, as the v2 scaffold runs it. Boxes labelled with an agent are model calls in a fresh context; the decisions, the start and end nodes, and the sample-test run are harness logic and call no model. The boxes absent from the original scaffold are the verifier of step 5 and the cut-off summarizer.

Pooled problem×pass agreement uses exact McNemar on the discordants, treating cells as independent.

6) Code and data availability: The evaluation set is the public hard split of LiveCodeBench release_v6 [18]; this study releases no new dataset. The orchestration scaffold (both versions), the corrected LiveCodeBench evaluator, the analysis and figure scripts that produce every number and figure in this paper, and all run outputs supporting the results - per-problem generations for every arm and condition, per-call `finish_reason` and token logs where instrumented, and the manager arms' workspace transcripts - are available at https://github.com/slee-persis/GVS5H.

III. Results

A. Pinned-Backend Arms, Five Passes

Table I and Fig. 2 report our headline condition: six models, each run five independent times on LCB-100 at a 128k cap with reasoning on. Every arm is served on a pinned backend and runs the v2 scaffold except Fable 5. The earlier runs of the OpenRouter-served set (Section III-D) cover more models, but only their 16k reasoning-off condition repeats; the rest are one pass each, through a gateway whose routing proved the main source of run-to-run noise.

All five models with both arms benefit from the manager. It also narrows the wide run-to-run spread for Luna and Qwen3.8-27B. Qwen3.8-27B gains most, its manager arm reaching Fable 5's single-call range here, and Opus-5's in the OpenRouter-served set. The direction matches that set's reasoning-on pattern: the smaller the model, the more the scaffold has to add.

B. What the Scaffold Costs

The manager uses more tokens by replacing one call with a plan, a brainstorm, up to ten worker rounds and a finalization pass. On average, the scaffold roughly triples the bill (Table II; supplementary Fig. S1 charts it per arm). Every arm priced here is at the same 128k output cap, Qwen3.8-27B's single call included (Appendix A).

The scaffold improves performance for a higher price. Fig. 3 plots price against accuracy: every model's manager arm gains accuracy over its single call, and costs more. On

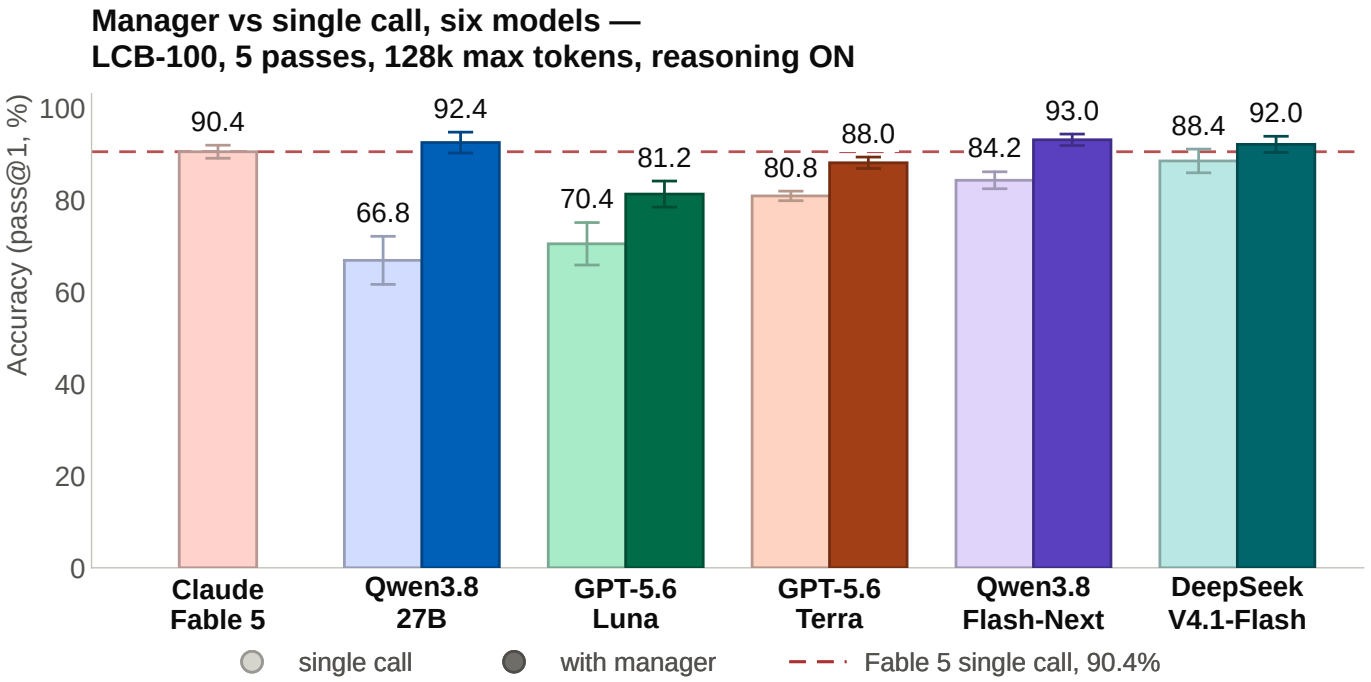


Fig. 2. Manager vs. single call. The scores from Table I in a graph. Bars are pass@1 on the same 100 problems, the line through each the 95% confidence interval (CI) across the 5 passes (t, df = 4). The left bar of each pair is the single call, light; the manager is beside it, dark; colour is the model. Fable 5 ran single-only, so it has one bar, which the dashed rule carries across the chart.

TABLE I
MANAGER VS. SINGLE CALL, FIVE PASSES

Model	Single call	With manager	Δ vs single	p	Δ vs Fable 5	95% bound	p
Claude Fable 5	90.4 ± 1.4	—	—	—	reference	—	—
GPT-5.6-Terra	80.8 ± 1.0	88.0 ± 1.2	$+7.2$	8.0×10^{-5}	-2.4	-5.8	0.88
GPT-5.6-Luna	70.4 ± 4.6	81.2 ± 2.8	$+10.8$	$< 2.5 \times 10^{-5}$	-9.2	-13.4	0.0013
Qwen3.8-27B	66.8 ± 5.2	92.4 ± 2.3	$+25.6$	$< 2.5 \times 10^{-5}$	$+2.0$	-0.8	0.88
Qwen3.8-Flash-Next	84.2 ± 1.8	93.0 ± 1.2	$+8.8$	$< 2.5 \times 10^{-5}$	$+2.6$	-0.1	0.56
DeepSeek-V4.1-Flash	88.4 ± 2.6	92.0 ± 1.8	$+3.6$	0.002	$+1.6$	-1.3	0.88

pass@1 on LCB-100 at a 128k cap, reasoning on; $\pm$ is a 95% t interval across the five passes (df = 4). Δ is in percentage points, against the model's own single call and against Fable 5's single call. Each p is a paired sign-flip permutation test, unit = problem ($n = 100$), Holm-corrected within its family of five; $<$ marks the permutation floor. **95% bound** is the one-sided lower bound on Δ vs Fable 5 over the same per-problem differences (t, df = 99): the largest deficit the data leave open.

average, the number of output tokens increases to about 3× baseline.

The base cost varies across models. A single-call pass on Qwen3.8-27B emits more than twenty times the output tokens of either OpenAI model. The total cost depends on the model's reasoning tendencies in addition to its per-token rate and scaffold.

1) Comparisons along the frontier: In many cases, the manager arm of a cheaper model can have a lower cost than a single call to an expensive model, while providing a higher accuracy. Most notably, Qwen3.8-Flash-Next scores at or above Fable 5 for a tenth of the price. Qwen3.8-27B also edges ahead of Fable 5 for a slightly lower price, with a much lower hardware requirement to host locally. Within the GPT-5.6 family, GPT-5.6-Luna with a manager matches GPT-5.6-Terra's single call on accuracy at under half the price, making a clear case for spending on the scaffold rather than the model.

C. The Effect of Truncation

Each model generation is cut off once its thinking and output exceed the token limit. In the manager case, a new, fresh round is started. In the single call run or the final call of a manager run, if no valid code was generated before the cutoff, the task is marked as a fail. Table III shows the frequency of these truncations.

Cap hits are rare across all models except the Qwen3.8 family, especially Qwen3.8-27B. Many runs that reached the cutoff still contained a complete solution earlier in their output or reasoning. Of the single arm's 35 empty cells, the manager solves 28, fails 7, and leaves none empty, worth 5.6 points. To investigate the effect of the cap, Qwen3.8-27B's single arm was allowed to generate up to 250k tokens. This higher limit raised the score by 3.2 points, to 70.0. Re-scoring each single arm over only the problem-passes where it emitted code bounds the

TABLE II
WHAT ONE PASS COST EACH ARM

Arm	Rate $/MTok in / out	In (MTok)	Out (MTok)	$/pass	$/solved	p vs single	p vs Fable 5
Qwen3.8-27B single	$0.21 / $2.55	0.0753	7.4247	$18.95	$0.28	—	—
Qwen3.8-27B manager	$0.21 / $2.55	1.5053	18.6277	$47.82	$0.52	1.5×10^{-4}	6.1×10^{-4}
Qwen3.8-Flash-Next single	$0.15 / $0.47	0.0753	5.7751	$2.73	$0.032	—	—
Qwen3.8-Flash-Next manager	$0.15 / $0.47	1.2930	11.8380	$5.76	$0.062	3.5×10^{-4}	3.6×10^{-7}
DeepSeek-V4.1-Flash single	$0.30 / $1.20	0.0680	2.8626	$3.46	$0.039	—	—
DeepSeek-V4.1-Flash manager	$0.30 / $1.20	1.4993	12.4873	$15.43	$0.17	1.6×10^{-4}	2.2×10^{-9}
GPT-5.6-Luna single	$0.20 / $1.20	0.0661	0.3299	$0.41	$0.006	—	—
GPT-5.6-Luna manager	$0.20 / $1.20	1.1686	1.0522	$1.50	$0.018	6.4×10^{-5}	1.3×10^{-6}
GPT-5.6-Terra single	$2 / $12	0.0661	0.2728	$3.41	$0.042	—	—
GPT-5.6-Terra manager	$2 / $12	1.1098	0.7911	$11.71	$0.13	1.6×10^{-4}	1.9×10^{-8}
Fable 5 single	$10 / $50	0.0899	1.2043	$61.11	$0.68	—	reference

MTok = million tokens. List rates: Qwen3.8-27B [28], Qwen3.8-Flash-Next [29], GPT-5.6-Luna and GPT-5.6-Terra [30], DeepSeek-V4.1-Flash [31], Fable 5 [32]. OpenAI uses short-context rates and Anthropic and DeepSeek use the base rates; no long-context tier, batch/off-peak discount, or prompt-caching multiplier is applied. Qwen and DeepSeek rates were read on Sep. 15, 2026.

List rate $\times$ the tokens it consumed. The two p columns test each manager arm against its own single call and against Fable 5's single call: Welch over the five passes, Holm-corrected across 11 comparisons. Qwen3.8-27B's single arm is the 128k cap-matched one; at 250k it costs $26.95/pass. Retried and discarded attempts are counted: they were generated and would be billed. The cheapest arm is GPT-5.6-Luna single at $0.41 a pass and the most accurate is Qwen3.8-Flash-Next manager at $5.76 - a 14$\times$ spread in price for +22.6 points.

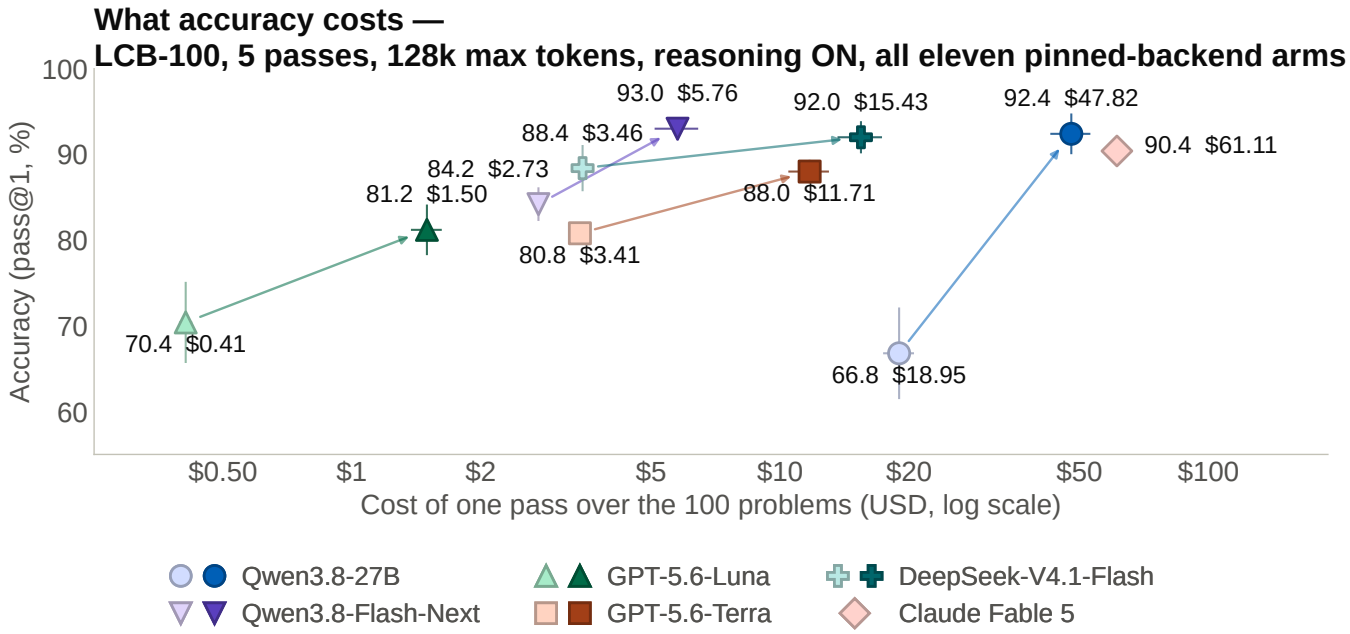


Fig. 3. Cost against accuracy. One point per arm; an arrow runs from the single call to the manager of each model that has both. x is dollars for one pass over the 100 problems (log scale), y is pass@1; the bar through each point is the 95% CI across the 5 passes (t, df = 4). Light fill = single call, dark = with manager; marker shape is the model.

effect model by model: the *emitted only* column of Table III carries that read for every arm, and puts Qwen3.8-27B's single call at 71.8. Fable 5 gains 1.7 points there, six of its nine empty cells being safety refusals rather than truncations.

D. The OpenRouter-Served Model Set

The five models below were served through the OpenRouter gateway and run on the original scaffold, so they are not directly comparable to the pinned-backend arms above. Table IV collects every condition.

TABLE III
CAP HITS AND EMPTY SOLUTIONS PER ARM

Model	Cap hits single	Cap hits manager	No code s / m	of which refusals	pass@1 s / m	Emitted only s / m	Cap
GPT-5.6-Terra	0 / 500	0 / 3,392	0 / 0	0	80.8 / 88.0	80.8 / 88.0	128k
GPT-5.6-Luna	0 / 500	0 / 3,498	0 / 0	0	70.4 / 81.2	70.4 / 81.2	128k
Qwen3.8-27B	150 / 500	108 / 3,235	35 / 0	0	66.8 / 92.4	71.8 / 92.4	128k[a]
Qwen3.8-27B (250k)	124 / 500	—	21 / —	0	70.0 / —	73.1 / —	250k[a]
Qwen3.8-Flash-Next	51 / 500	92 / 3,141	20 / 0	0	84.2 / 93.0	87.7 / 93.0	128k
DeepSeek-V4.1-Flash	11 / 500	49 / 3,525	10 / 1	0	88.4 / 92.0	90.2 / 92.2	128k
Claude Fable 5	3 / 500	—	9 / —	6	90.4 / —	92.1 / —	128k

[a] Qwen3.8-27B's single arm was run at 250k and cap-matched back to 128k, where a cap hit is a completion reaching 128,000 output tokens.

A *cap hit* is a generation that stopped at the token limit (`finish_reason=length`). *No code* counts any run containing no code, due to a cap hit or other reason.

TABLE IV
LCB-100 PASS@1 (%), SINGLE → MANAGER

Model	Params	128k · ON (1 pass)	128k · OFF (1 pass)	16k · OFF (×5)
Opus-5	n/a	88 → 95 (**+7**)	–	–
Kimi-K3	∼2.8T	86 → 86 (+0)	33 → 76 (**+43**)	33.4 → 65.0 (**+31.6**)
MiniMax-M3	428B	62 → 69 (**+7**)	26 → 40 (**+14**)	22.2 → 33.2 (**+11.0**)
Qwen3.6-35B	35B	26 → 44 (**+18**)	35 → 27 (−8)	28.2 → 27.4 (−0.8)
Qwen3.5-9B	9B	*unusable*	18 → 21 (+3)	15.0 → 22.6 (**+7.6**)

For the OpenRouter-served models on the original scaffold. "–" = not run; Qwen3.5-9B reasoning-on returns reasoning-only replies and is unusable. Scores are from the corrected grader (supplementary Section I): the four LCB-100 problems whose reference answer is not unique are judged by the original contest rule, and submissions run as real subprocesses so `sys.stdout.buffer` behaves as it does on the contest judge. A bolded delta clears the ∼4.5 point pass-to-pass band. The eleven pinned-backend, five-pass arms are reported separately in Section III and are not pooled here, because both the serving path and the scaffold version differ (Section II-C).

1) Reasoning on: Opus-5 scores at 88 single → 95 manager, the highest of all tested scenarios. However, smaller models tend to gain more from the manager (Fig. 4(a); supplementary Fig. S2 by scale).

2) Reasoning off: We ran five independent passes per condition at a 16k cap plus a one-pass 128k control. Run-to-run spread across the five passes is small. Kimi-K3 exhibits a particularly large gain, +31.6 points (Fig. 4(b); Fig. 5 by scale). The 128k control largely eliminates truncation and reproduces the ordering (panel (c); supplementary Fig. S3).

IV. DISCUSSION

Table V collects one transcript per model in which the single call failed and the manager arm solved the problem. Three mechanisms recur. The scaffold commits the reduction or the bounding argument to disk before any code is written, so the next call builds on it instead of re-deriving it. It splits coupled objectives into pieces that are implemented separately. And it bounds each call, so a model that would otherwise talk past its output cap gets its work written down. In two cases the manager's solution is also markedly smaller than the failed single-call one.

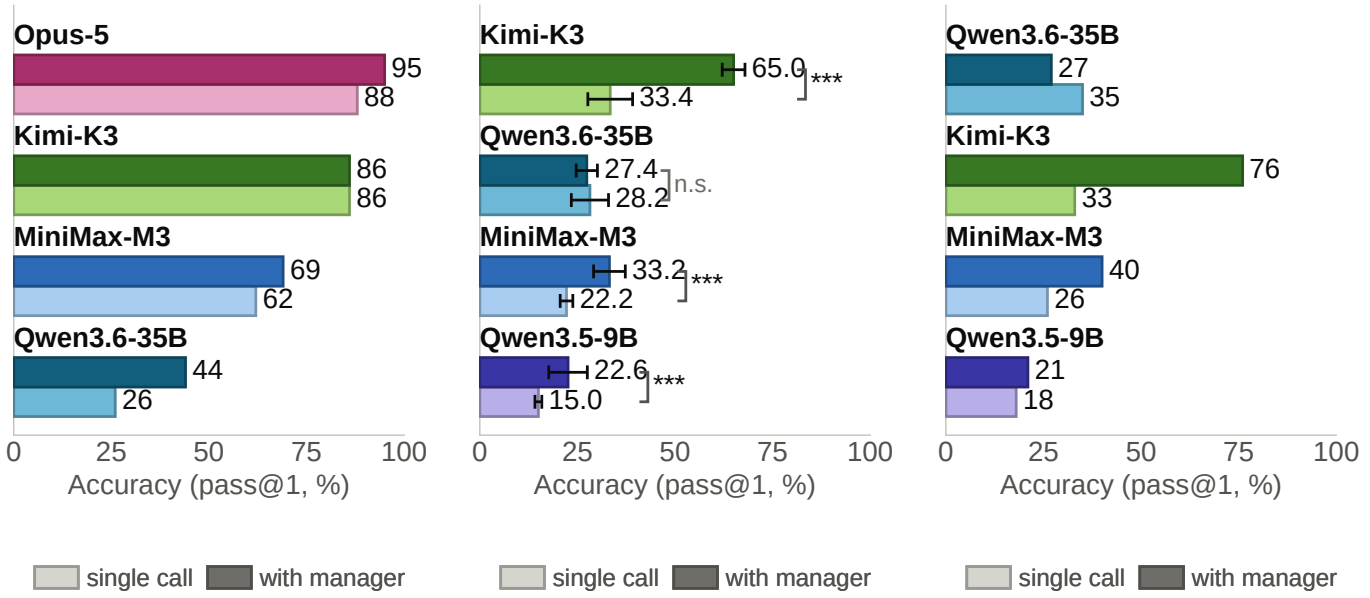
Truncation is a second place the scaffold earns its keep. State lives in `plan.md`, `notes.md` and `solution.py` rather than in one ever-growing transcript, and each worker sees a curated view of it - plan, notes, current solution, one task - instead of the full history. The workspace is external memory that the token cap cannot truncate.

That matters because a generation left to run to a very large cap does not merely risk being cut off; it can degrade the model. Eight of the Qwen3.8-27B single calls became unstable during reasoning and used up the full 250k budget without emitting code - on `abc399_e`, the model repeated the phrase "`Alphabet 5 with a<->b, c<->d, e<->e fixed, and one letter? no.`" 7,743 times. This is the mechanism behind the otherwise odd finding of Section III-C that most cut-off generations still contain a complete solution: the model had reached an answer and then talked past the cap re-checking it. Bounding each call keeps the work far from the cap and gets it onto disk.

The same argument should extend beyond this benchmark, though we do not test it. A task whose inputs or intended output do not fit in one call - a codebase larger than the context window, a deliverable longer than the output ceiling - is not solvable in one call at any reasoning effort, whereas a sequence of scoped tasks can make progress.

The manager helps most where the base model is least able to organize its own reasoning, and that weakness takes two forms. The first is having no internal planning stage at all: with reasoning disabled a single call jumps straight to code, and the scaffold supplies a planning space, on disk and across calls, that the model would not otherwise use (Fig. 4(c)). The second is reasoning that runs away: Qwen3.6-35B with reasoning on spends its whole budget deliberating, and gains more here than in any of its other conditions (panel (a)). Where a model can both plan internally and stop on its own, the scaffold has less to add.

The manager is not free: its deliberation can settle on a worse answer than a single call would have produced. On LCB 3765 (Qwen3.6-35B, 128k, reasoning off) the ideation stage identified the same optimization the single call had used, talked itself out of it as too error-prone, and committed instead to



(a) 128k max tokens, reasoning on, one pass per condition.

(b) 16k max tokens, reasoning off, five passes per condition.

(c) 128k max tokens, reasoning off, one pass per condition.

Fig. 4. Manager vs. single call, three conditions. Models run top to bottom by single-call score, best first, so a model's row differs between panels; the top bar of each pair is the manager. Fill: light = single call (one call, no tools), dark = with manager; Δ = manager $-$ single, in percentage points. The line through a bar is the 95% CI across its passes, where the condition ran more than one. A panel prints its two bar values and nothing else; the Δ and Tukey honestly significant difference (HSD) group letter quoted below per model are what it leaves out (shared letter = not significantly different across the models of that condition, $\alpha = 0.05$). (a) 128k max tokens, reasoning on - effort:high for Kimi-K3 and Opus-5, a 20k reasoning budget requested for Qwen3.6-35B and MiniMax-M3 that providers often did not honor. One pass per condition, so no per-model p. Non-empty completions per 100 (single→manager): Opus-5 97→100, Kimi-K3 97→100, MiniMax-M3 92→93, Qwen3.6-35B 34→68. Empty or truncated output scores as a fail, so Δ partly tracks emit rate. Opus-5 Δ+7.0 (group ab), Kimi-K3 Δ+0.0 (group b), MiniMax-M3 Δ+7.0 (group ab), Qwen3.6-35B Δ+18.0 (group a). (b) 16k max tokens, reasoning off, five passes per condition; the line through each bar is the 95% CI across the five passes (t, df = 4). Each Δ is a paired sign-flip permutation test, unit = problem (n = 100), Holm-corrected across the 4 models; the bracket beside a pair grades its p: * $p < .05$, ** $p < .01$, *** $p < .001$, n.s. otherwise. Kimi-K3 Δ+31.6 (group a), Qwen3.6-35B Δ-0.8 (group c), MiniMax-M3 Δ+11.0 (group b), Qwen3.5-9B Δ+7.6 (group bc). (c) 128k max tokens, reasoning off (ESCALATION_OR_REASONING=none), one pass per condition, so no per-model p. Non-empty completions per 100 (single→manager): Qwen3.6-35B 85→95, Kimi-K3 96→100, MiniMax-M3 93→100, Qwen3.5-9B 79→92. Qwen3.6-35B Δ-8.0 (group c), Kimi-K3 Δ+43.0 (group a), MiniMax-M3 Δ+14.0 (group b), Qwen3.5-9B Δ+3.0 (group bc).

an asymptotically slower plan that a worker then implemented with a further bug - compact, correct code disturbed rather than helped by decomposition. Qwen3.6-35B with reasoning off is the standout loser on this pattern, negative at both caps.

Two inexpensive, training-free additions could address the two halves of that failure, the plan that was wrong and the implementation that went unchecked. First, fresh-perspective workers: every worker inherits the notes and the current solution, so a wrong early approach anchors everything downstream. Spawning some workers with the raw problem statement and no prior context would give the manager an independent attempt to compare against, protecting the tight single-call solution the scaffold otherwise disturbs - the intuition behind sampling-and-voting and debate [13], [14]. Second, verification instead of trust: the manager currently accepts a worker's report on the basis of a sanity check against the problem's own samples, so a confidently-wrong solution propagates unchecked and can overwrite a correct intermediate answer. Running each candidate against a generated test suite before acceptance would catch regressions of this kind.

A. Limitations

- Five models ran through OpenRouter's multi-provider routing. OpenRouter does not guarantee one quantization or build per model. Many providers also intermittently stalled, dropped the response mid-stream, returned error objects, cut off output below the requested cap, or returned reasoning-only replies, resulting in a time-varying confound.
- Only one pass was run for the reasoning-on and 128k reasoning-off comparisons to limit API cost, so their results are a point estimate only.
- Fable 5 was run single-only due to its high cost, so it is unknown how much improvement its manager arm would gain.
- Fable 5 returned `stop_reason=refusal` on 6 of 500 problem-passes, resulting in a slight score reduction of 0-1.2 points.
- All results are based on code generation. Math and knowledge benchmarks (AIME, MATH-500, GPQA, HLE) were explored but abandoned due to saturation at near perfect scores.

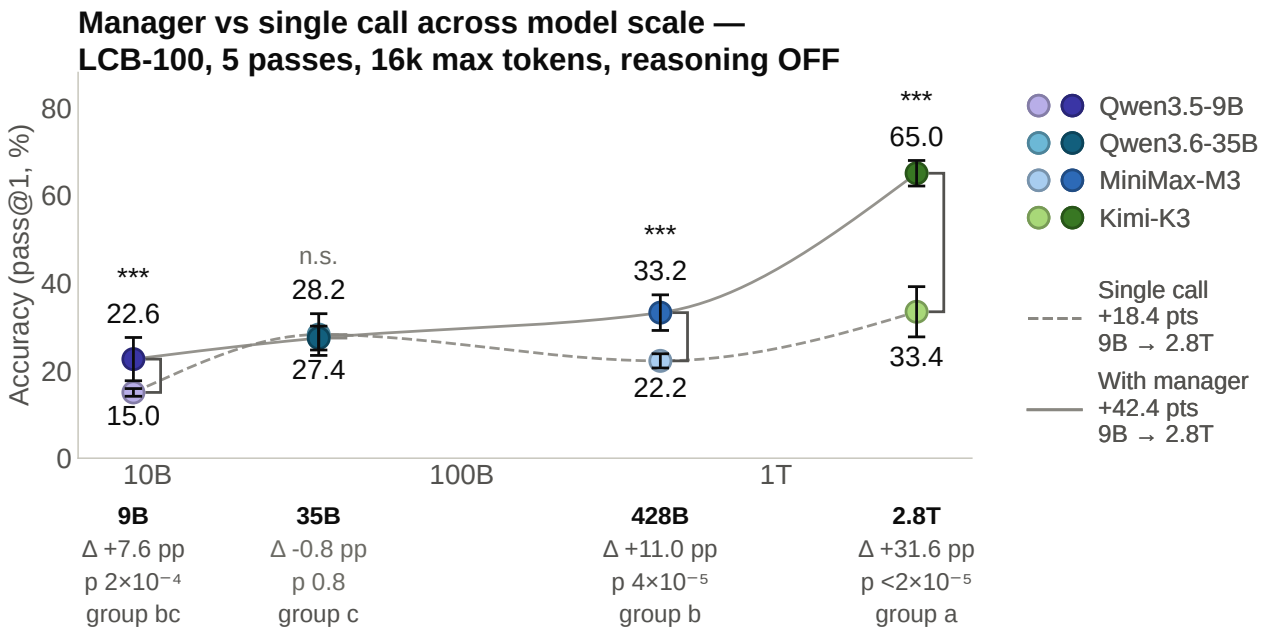


Fig. 5. Accuracy by model scale, reasoning off, 16k, five passes. The same numbers as Fig. 4(b), against parameter count; the line through each mark is the 95% CI across the five passes. Dashed = single call, solid = with manager, and the bracket at a pair grades its Holm-corrected p; the block under each tick gives that model's Δ, p and Tukey group, and the legend each arm's gain from 9B to 2.8T.

V. Conclusion

The GVS5H scaffold improves the coding accuracy of each pinned-backend model run in both arms. These results provide a baseline against which more complex systems involving heterogeneous models or code verification can be compared. The scaffold, despite increasing token use and hence cost compared to a model's single call, often remains cheaper than frontier models while providing a higher accuracy. Transcript analysis attributes the gain to decomposition and to context that persists across calls. The result is conditional - some configurations result in no improvement or regression. However, in many cases, the orchestration is a cost-effective substitute for model scale.

Appendix A
Benchmark and Protocol in Full

A. Caps

The 128k figure is a per-call `max_tokens` bound, not a context-window limit; for Opus-5, Fable 5 and the GPT-5.6 family it is also the provider's hard ceiling on one response [26], [27]. Locally served Qwen3.8-27B has no such ceiling; it is bounded by its 262,144-token context window [21] shared by prompt and output, which puts the 250k setting of Section III-C near its largest possible output.

B. Reasoning

The output cap and the scaffold are matched across the eleven pinned-backend arms; reasoning depth is not, because no two of these providers expose the same control over it. Qwen3.8-27B, on our own vLLM, is sent no `reasoning_effort`. The Qwen chat template leaves reasoning on and unbudgeted, bounded only by the 128k `max_tokens`, which on this stack covers reasoning and answer together. The two GPT-5.6 models are likewise sent none, leaving them at OpenAI's model default. Fable 5 runs at `thinking: adaptive` with `display: summarized`, and `output_config.effort` at `high`, the API default [33].

C. Instrumentation

The two 128k-cap conditions log every call's `finish_reason` and completion-token count, and give each problem a final status in {`ok`, `truncated`, `empty_stop`, `empty`, `error`}. The 16k × 5-pass runs predate this and record only the extracted code and its pass/fail. An empty code field scores as a failure.

D. Cap-matching

One arm is not natively cap-matched to its partner: Qwen3.8-27B's single call was generated at 250k while its manager arm ran at 128k (Section III-C). To compare them at one cap we replay the stored generations rather than re-run the model: each is truncated to 128,000 output tokens, the solution re-extracted from that prefix, and the result scored on the same evaluator as every other cell (supplementary Section I):

- **Truncation is token-exact**, using the serving stack's own tokenizer instead of a character-count approximation.
- **What is truncated is the whole output stream, reasoning then answer**, because that is what the original cap bounded. Where the cut lands decides what the extractor sees: past the reasoning it sees a truncated answer; inside the reasoning the answer does not exist at all, and the harness's empty-content fallback hands it the truncated reasoning instead. (Section III-C).

TABLE V
ONE TRANSCRIPT PER MODEL WHERE THE SINGLE CALL FAILED AND THE MANAGER ARM SOLVED THE PROBLEM

Model (condition)	Problem	How the single call failed	What the scaffold added
Qwen3.5-9B (128k, off)	abc385_d	Timed out: a naïve per-segment scan of a path simulation.	The brainstorm flagged in writing that the check is $O(N \cdot M) \approx 4 \times 10^{10}$ and prescribed a range-query structure before any code - the up-front planning a single pass skips (Section IV).
Qwen3.6-35B (on)	abc394_f	Cut off mid-reasoning at 32,768 tokens; returned nothing.	Solved in one worker cycle: the brainstorm fixed the structural insight (an "alkane" subtree needs a degree-4 center and ≥ 5 vertices); the worker used an iterative depth-first search (DFS) to avoid a recursion-depth failure. The same 32k clamp was hit 117 times across this model's manager calls, but with several attempts per problem one clamped call no longer costs the problem.
MiniMax-M3 (on)	3701	Printed nothing on `cdcd`: the lexicographic-reconstruction branch was broken.	Split the two coupled objectives across worker cycles - one worker on the forward cost dynamic program (DP) over capped run-lengths, a separate worker on a suffix DP used purely for the lexicographic reconstruction.
Kimi-K3 (128k, off)	3687	Over-counted the longest unique-value path (9 and 3 where the answers were 6 and 2), never shrinking its window on a repeated value.	The brainstorm wrote the reduction into `notes.md` first - longest substring without repeating characters along each root-to-leaf path, window start via last-seen depths, $O(n)$ DFS. Only then did a worker implement it.
GPT-5.6-Terra (on)	3688	4,615 characters implementing a Segment-Tree-Beats-shaped structure indexed by candidate value, and wrong.	The plan named the difficulty in advance - the state must be compressed rather than maintaining all distinct values per index. Nine calls later, the manager arm returned 1,860 characters over one standard segment tree. The contribution is *subtraction*. Manager-only win in four of five passes.
GPT-5.6-Luna (on)	abc397_e	Carried a *set* of unfinished path lengths per subtree across 4,092 characters, a general state it never closed.	The brainstorm established the bounding lemma (at most two unfinished paths at any vertex), after which a worker implemented the postorder scan directly in 2,166 characters over five calls. Manager-only win in two of five passes.
Qwen3.8-27B (on)	abc397_g	Spent all 250,000 tokens on edge-case self-interrogation and never emitted a program - the runaway-reasoning failure mode of Section IV in pure form - though the reasoning had already reached the min-cut formulation.	Committed that same reduction to `plan.md` as a plan, then had a worker implement it: five calls, 3,548 characters, manager-only win in all five passes. The scaffold does not supply the insight; it forces it onto disk before the budget runs out.
Opus-5 (on)	abc388_e	The feasibility test over-counted, reporting 225 pairs where 220 is optimal.	Recorded the justification in `notes.md` first - an exchange argument, plus monotonicity of feasibility in K licensing a binary search - then had one worker implement it and a second harden it.

(Section IV). LCB problem IDs as in release_v6; call counts are manager-arm model calls. Character counts are of the emitted program.

TABLE VI
PROBLEM-PASSES WON BY ONE ARM ALONE

Model	Manager only	Single call only	Problem-passes	Exact McNemar p
GPT-5.6-Terra	47	11	500	2.0×10^{-6}
GPT-5.6-Luna	72	18	500	8.1×10^{-9}
Qwen3.8-27B	134	6	500	1.4×10^{-32}
Qwen3.8-Flash-Next	49	5	500	3.9×10^{-10}
DeepSeek-V4.1-Flash	22	4	500	5.3×10^{-4}

Of the 500 problem-passes per model (100 problems $\times$ 5 passes), the number the manager arm solved and the single call did not, against the number the single call solved and the manager did not. The gains are not compensating wins and losses. The test is exact McNemar on those discordant pairs (Section II-C).

- **The cap is invisible to the model.** This arm ran on our own vLLM, where `max_tokens` is enforced by the server: it never enters the prompt, and generation simply cuts off at the limit, abruptly rather than winding up (124 of the 500 calls end at `finish_reason=length`). Nor is this local to vLLM: making a budget visible to a model takes a separate, opt-in mechanism such as Anthropic's `task_budget`, while `max_tokens` stays an enforced ceiling that truncates the response [33]. The replay therefore does not differ from a genuine re-run at 128k.

APPENDIX B
ADDITIONAL RESULTS

Table VI counts, for each model run in both arms, the problem-passes that one arm solved and the other did not.

ACKNOWLEDGMENT

Generative artificial intelligence is the object of study in this work: every experimental result is a measurement of

language-model output, produced by the scaffold and baselines described in Section II and released with the code. Separately, generative language-model based tools assisted with reviewing the manuscript, with drafting this disclosure, with implementing the orchestration harness and the analysis and plotting scripts, and with literature search. Generative artificial intelligence tools were not used to generate, impute or alter any experimental result, and all reported numbers are computed by the released scripts directly from the stored run outputs. The authors have reviewed all artificial intelligence-assisted work: the harness and analysis code was read and tested by the authors, the corrected evaluator of supplementary Section I was found and verified by manual inspection of transcripts, and every citation was checked against its primary source. The authors take responsibility for the final content of this work, including text, claims and artifacts produced with the aid of generative artificial intelligence.

REFERENCES

[1] Sakana AI, "Sakana Fugu technical report," 2026. [Online]. Available: https://doi.org/10.48550/arXiv.2606.21228

[2] S. Nielsen *et al.*, "Learning to orchestrate agents in natural language with the Conductor," in *International Conference on Learning Representations*, 2026.

[3] I. Ong *et al.*, "RouteLLM: Learning to route LLMs with preference data," in *International Conference on Learning Representations*, 2025.

[4] S. Hong *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *International Conference on Learning Representations*, 2024.

[5] C. Qian *et al.*, "ChatDev: Communicative agents for software development," in *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, 2024.

[6] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "CAMEL: Communicative agents for "mind" exploration of large language model society," in *Advances in Neural Information Processing Systems*, 2023.

[7] D. Huang *et al.*, "AgentCoder: Multi-agent-based code generation with iterative testing and optimisation," 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2312.13010

[8] M. A. Islam, M. E. Ali, and M. R. Parvez, "MapCoder: Multi-agent code generation for competitive problem solving," in *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, 2024.

[9] Q. Wu *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," in *Conference on Language Modeling*, 2024.

[10] J. Xu *et al.*, "TRINITY: An evolved LLM coordinator," in *International Conference on Learning Representations*, 2026.

[11] M. Wei, X. Chen, X. Niu, and S. Chen, "ARIADNE: Agentic reward-informed adaptive decision exploration via blackboard-driven MCTS for competitive program generation," 2026. [Online]. Available: https://doi.org/10.48550/arXiv.2605.02431

[12] B. Han and S. Zhang, "Exploring advanced LLM multi-agent systems based on blackboard architecture," 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2507.01701

[13] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving factuality and reasoning in language models through multiagent debate," in *International Conference on Machine Learning*, 2024.

[14] J. Li, Q. Zhang, Y. Yu, Q. Fu, and D. Ye, "More agents is all you need," *Transactions on Machine Learning Research*, 2024.

[15] J. Wang, J. Wang, B. Athiwaratkun, C. Zhang, and J. Zou, "Mixture-of-agents enhances large language model capabilities," in *International Conference on Learning Representations*, 2025.

[16] J. Wang *et al.*, "Reasoning in token economies: Budget-aware evaluation of LLM reasoning strategies," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, 2024.

[17] D. Tran and D. Kiela, "Single-agent LLMs outperform multi-agent systems on multi-hop reasoning under equal thinking token budgets," 2026. [Online]. Available: https://doi.org/10.48550/arXiv.2604.02460

[18] N. Jain *et al.*, "LiveCodeBench: Holistic and contamination free evaluation of large language models for code," 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2403.07974

[19] Qwen Team, "Qwen3.5: Towards native multimodal agents," 2026, model card at https://huggingface.co/Qwen/Qwen3.5-9B; accessed 17 August 2026. [Online]. Available: https://qwen.ai/blog?id=qwen3.5

[20] ——, "Qwen3.6-35B-A3B model card," 2026, accessed 17 August 2026. [Online]. Available: https://huggingface.co/Qwen/Qwen3.6-35B-A3B

[21] ——, "Qwen3.8-27B model card," 2026, accessed 25 August 2026. [Online]. Available: https://huggingface.co/Qwen/Qwen3.8-27B

[22] ——, "Qwen3.8-Flash-Next model card," 2026, accessed 15 September 2026. [Online]. Available: https://huggingface.co/Qwen/Qwen3.8-Flash-Next

[23] DeepSeek-AI, "DeepSeek-V4.1-Flash model card," 2026, accessed 15 September 2026. [Online]. Available: https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash

[24] MiniMax, "MiniMax-M3," 2026, model card at https://huggingface.co/MiniMaxAI/MiniMax-M3; accessed 19 August 2026. [Online]. Available: https://www.minimax.io/blog/minimax-m3

[25] Moonshot AI, "Kimi K3," 2026, quickstart at https://platform.kimi.ai/docs/guide/kimi-k3-quickstart; accessed 19 August 2026. [Online]. Available: https://www.kimi.ai/blog/kimi-k3

[26] Anthropic, "Claude models overview," 2026, accessed 14 August 2026. [Online]. Available: https://platform.claude.com/docs/en/about-claude/models/overview

[27] OpenAI, "GPT-5.6 model documentation," 2026, model card at https://developers.openai.com/api/docs/models/gpt-5.6-luna; accessed 19 August 2026. [Online]. Available: https://developers.openai.com/api/docs/guides/latest-model

[28] OpenRouter, "Qwen3.8-27B," 2026, accessed 15 September 2026. [Online]. Available: https://openrouter.ai/qwen/qwen3.8-27b

[29] ——, "Qwen3.8 Flash," 2026, accessed 15 September 2026. [Online]. Available: https://openrouter.ai/qwen/qwen3.8-flash

[30] OpenAI, "API pricing," 2026, accessed 25 August 2026. [Online]. Available: https://developers.openai.com/api/docs/pricing

[31] DeepSeek, "Models & pricing," 2026, accessed 15 September 2026. [Online]. Available: https://api-docs.deepseek.com/quick_start/pricing

[32] Anthropic, "Pricing," 2026, accessed 25 August 2026. [Online]. Available: https://platform.claude.com/docs/en/about-claude/pricing

[33] ——, "Effort, thinking and task budgets," 2026, with https://platform.claude.com/docs/en/build-with-claude/thinking and https://platform.claude.com/docs/en/build-with-claude/task-budgets; accessed 25 August 2026. [Online]. Available: https://platform.claude.com/docs/en/build-with-claude/effort